CSS3344 Mobile Application Development

Lab 02: UI Fundamentals and Layout

Lab Objectives:

By the end of this practical session, you will be able to demonstrate:

1. **Layout Decomposition & Trees:** Diagramming UI components into Rows, Columns, Containers, and modular sub-widgets.
2. **State & Orientation:** Handling StatelessWidget vs StatefulWidget and adapting layouts dynamically for Portrait/Landscape orientations.
3. **UI Components & Customization:** Scrollable ListViews, Pickers (Material & Cupertino)

Part 1: UI Concepts & Design systems

Flutter builds its UI out of widgets. Widgets describe what their view should look like given their current configuration and state. Flutter structures its UI layer into two main design paradigms:

- **Material Design *(package:flutter/material.dart):*** Google's design system providing pre-styled widgets like Scaffold, AppBar, ElevatedButton, and Switch.
- **Cupertino *(package:flutter/cupertino.dart):*** Apple's iOS-style visual language featuring components like CupertinoPageScaffold, CupertinoNavigationBar, and CupertinoDatePicker.
- **Core Widgets *(package:flutter/widgets.dart):*** Style-neutral base widgets (Container, Column, Row, Text, Image) used when building custom design systems.

When to use widgets?

- Building a custom design system: If your application uses a bespoke visual language that doesn't strictly follow Material Design or Apple Human Interface Guidelines, build on top of widgets rather than overriding opinionated design system defaults.
- Creating design-agnostic packages: If you publish a reusable component or utility package on pub.dev, importing *package:flutter/widgets.dart* ensures your package can be used in both Material and Cupertino applications without introducing unnecessary dependencies.
- Embedded and non-standard devices: For automotive, kiosks, or embedded displays where platform conventions don't apply and minimal bundle size is critical.

Layout Construction Workflow:

- **Diagram the Layout**: Identify parent-child relationships before writing code ("Measure twice, cut once").
- **Extract Sub-Widgets**: Divide complex UIs into smaller custom widgets (e.g., TitleSection, ButtonSection) to maintain clean code and optimized rendering trees.
- **Compose in Body:** Nest smaller widgets inside scrollable layout containers like ListView or SingleChildScrollView.

Note: Please reuse the previous Lab01 project for these tasks.

Part 2: Guided Learning

Exercise 2.1: Deconstructing a Section Widget

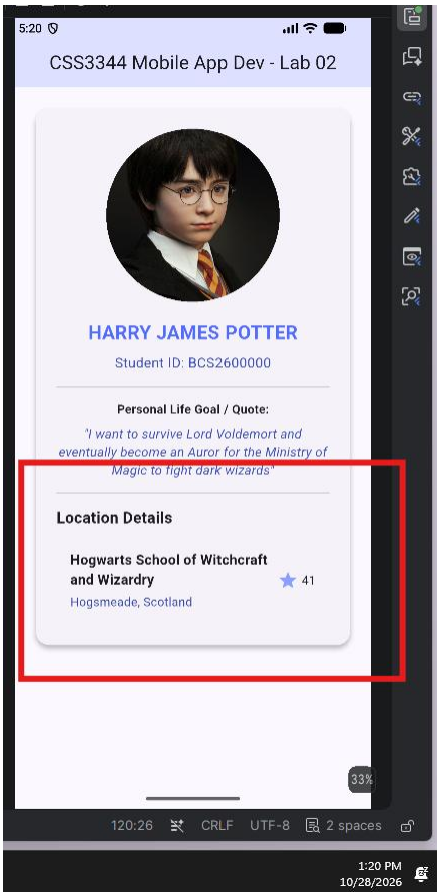
The following pattern demonstrates decomposing a layout section into a clean, reusable widget with flex distribution: *TitleSection*, *Expanded*, and passing data via Widget Parameters.

Base code:

```
class TitleSection extends StatelessWidget {
  final String name;
  final String location;
  final int starCount;

  const TitleSection({
    super.key,
    required this.name,
    required this.location,
    required this.starCount,
  });
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(24.0),
      child: Row(
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  name,
                  style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 18),
                ),
                const SizedBox(height: 4),
                Text(
                  location,
                  style: TextStyle(color: Colors.grey[600]),
                ),
              ],
            ),
          ),
          Icon(Icons.star, color: Colors.red[500]),
          const SizedBox(width: 4),
          Text('$starCount'),
        ],
      ),
    );
  }
}
```

Expected Output:



Exercise 2.2: Orientation & Adaptive Layouts

Organise the decomposed widgets created (StudentDetailsSection, TitleSection, etc.) inside an OrientationBuilder. Using OrientationBuilder allows your decomposed sections to arrange horizontally in landscape mode and vertically in portrait mode, preventing overflow errors.

- **In Portrait Mode:** Stack StudentDetailsSection and TitleSection vertically
- **In Landscape Mode:** Place StudentDetailsSection and TitleSection side-by-side

Base code:

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Adaptive Layout')),
    body: OrientationBuilder(
      builder: (context, orientation) {
        return orientation == Orientation.portrait
            ? _buildPortraitView()
            : _buildLandscapeView();
```

```
      },
    ),
  );
}

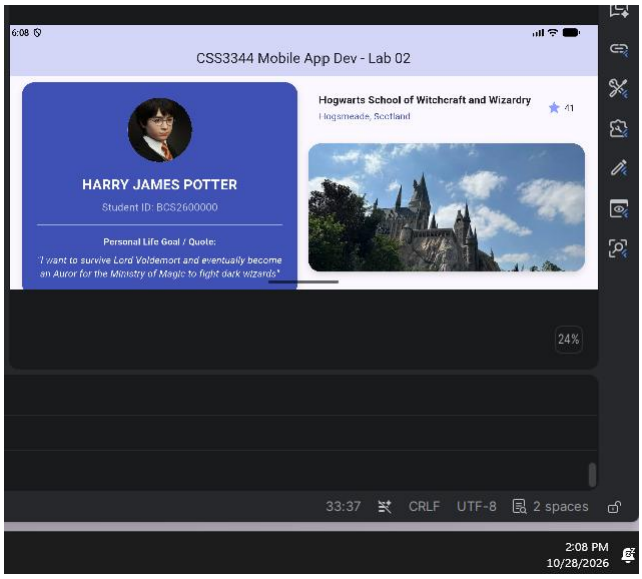
Widget _buildPortraitView() {
  return ListView(
    children: const [
      ImageSection(imagePath: 'assets/images/lake.jpg'),
      TitleSection(name: 'Oeschinen Lake', location: 'Kandersteg, Switzerland', starCount: 41),
      ActionButtonsSection(),
      TextSection(),
    ],
  );
}

Widget _buildLandscapeView() {
  return Row(
    children: [
      Expanded(child: ImageSection(imagePath: 'assets/images/lake.jpg')),
      Expanded(
        child: ListView(
          children: const [
            TitleSection(name: 'Oeschinen Lake', location: 'Kandersteg, Switzerland', starCount: 41),
            ActionButtonsSection(),
            TextSection(),
          ],
        ),
      ),
    ],
  );
}
```

Expected output: Portrait mode



Expected output: Landscape mode



2.2 Pickers: Material vs. Cupertino

Incorporate all four pickers (Material Date Picker, Cupertino Date Picker in a bottom sheet, Material DropdownButton, and Cupertino Wheel Picker in a bottom sheet).

Base code:

Material Date Picker

```
Future<void> _selectMaterialDate(BuildContext context) async {
  final DateTime? picked = await showDatePicker(
    context: context,
    initialDate: DateTime.now(),
    firstDate: DateTime(1950),
    lastDate: DateTime.now(),
  );
  if (picked != null) {
    setState(() {
      _selectedDate = picked;
    });
  }
}
```

Cupertino Date Picker (Modal Sheet)

```
void _showCupertinoDatePicker(BuildContext context) {
  showCupertinoModalPopup(
    context: context,
    builder: (_) => Container(
      height: 240,
      color: CupertinoColors.systemBackground.resolveFrom(context),
      child: CupertinoDatePicker(
        mode: CupertinoDatePickerMode.date,
        initialDateTime: DateTime.now(),
        onDateTimeChanged: (DateTime newDate) {
          setState(() => _selectedDate = newDate);
        },
      ),
    ),
  );
}
```

Selection Pickers: Material DropdownButton

```
DropdownButton<String>(
  value: _selectedRole,
  items: ['Student', 'Developer', 'Designer'].map((String role) {
    return DropdownMenuItem<String>(
```

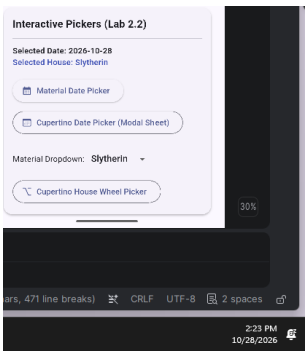
```
      value: role,
      child: Text(role),
    );
  }).toList(),
  onChanged: (String? newValue) {
    if (newValue != null) {
      setState(() => _selectedRole = newValue);
    }
  },
)
```

Selection Pickers: Cupertino Picker (Wheel)

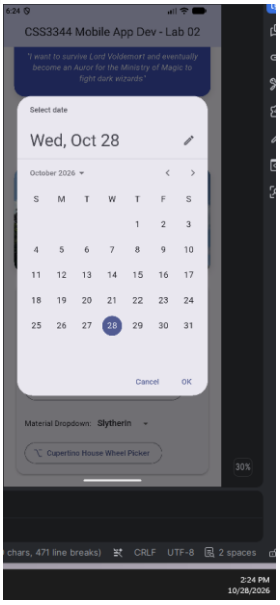
```
void _showCupertinoRolePicker(BuildContext context) {
  final List<String> roles = ['Student', 'Developer', 'Designer'];
  showCupertinoModalPopup(
    context: context,
    builder: (_) => Container(
      height: 200,
      color: CupertinoColors.systemBackground.resolveFrom(context),
      child: CupertinoPicker(
        itemExtent: 32,
        onSelectedItemChanged: (int index) {
          setState(() => _selectedRole = roles[index]);
        },
        children: roles.map((role) => Center(child: Text(role))).toList(),
      ),
    ),
  );
}
```

Expected Outputs:

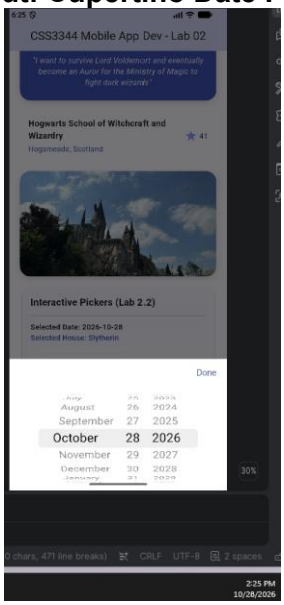
Output: All four Pickers



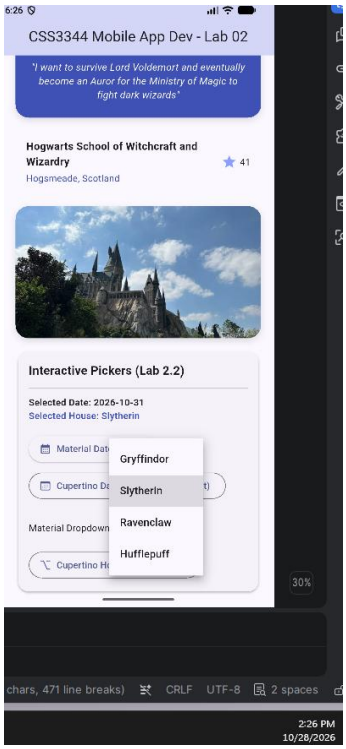
Output: Material Date Picker



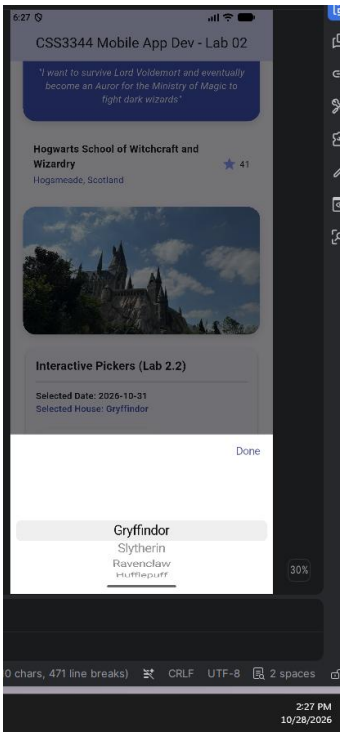
Output: Cupertino Date Picker



Output: Material Dropdown



Output: Cupertino House Wheel Picker



Lab Submission Checklist

Before submitting your lab assignment to Google Classroom, ensure you complete the following:

- [ ] Source Code link: Upload your complete Flutter project folder (exclude .dart_tool, build, and .idea folders to save space) into a GitHub, set it to **public**.
- [ ] Screenshots: Capture and upload clear screenshots showing:
    - Your machine date and time
    - Active Date Picker dialog/sheet on screen.
- [ ] Compile all screenshots and paste GitHub link into PDF

Source:

1. https://docs.flutter.dev/ui/layout
2. https://docs.flutter.dev/ui/widgets

Lab 02 Ends
